

ROSA: A Knowledge-based Solution for Robot Self-Adaptation

Gustavo Rezende Silva^{1,*}, Juliane Päßler², S. Lizeth Tapia Tarifa²,
Einar Broch Johnsen², and Carlos Hernández Corbato¹

¹*Delft University of Technology, Delft, Netherlands*

²*University of Oslo, Oslo, Norway*

Correspondence*:

Gustavo Rezende Silva

g.rezendesilva@tudelft.nl

2 ABSTRACT

3 Autonomous robots must operate in diverse environments and handle multiple tasks despite
4 uncertainties. This creates challenges in designing software architectures and task decision-
5 making algorithms, as different contexts may require distinct task logic and architectural
6 configurations. To address this, robotic systems can be designed as self-adaptive systems
7 capable of adapting their task execution and software architecture at runtime based on their
8 context. This paper introduces ROSA, a novel knowledge-based framework for RObot Self-
9 Adaptation, which enables task-and-architecture co-adaptation (TACA) in robotic systems. ROSA
10 achieves this by providing a knowledge model that captures all application-specific knowledge
11 required for adaptation and by reasoning over this knowledge at runtime to determine when
12 and how adaptation should occur. In addition to a conceptual framework, this work provides
13 an open-source ROS 2-based reference implementation of ROSA and evaluates its feasibility
14 and performance in an underwater robotics application. Experimental results highlight ROSA's
15 advantages in reusability and development effort for designing self-adaptive robotic systems.

16 **Keywords:** self-adaptation, knowledge representation, underwater vehicle, robotics

1 INTRODUCTION

17 A current challenge in robotics is designing software architectures and task decision-making algorithms
18 that enable robots to autonomously perform multiple tasks in diverse environments while handling internal
19 and environmental uncertainties. This challenge arises because different contexts may demand distinct task
20 logic and architectural configurations. At runtime, certain actions may become unfeasible, requiring the
21 robot to adapt its task execution to ensure mission completion. For example, a robot navigating through an
22 environment might run out of battery during its operation, requiring it to adapt its task execution to include
23 a recharge action. Additionally, actions may require different architectural configurations depending on the
24 context. For example, a navigation action that relies on vision-based localization cannot be executed in
25 environments without lights but could potentially be executed with an alternative architectural configuration
26 that employs a localization strategy based on lidar. This becomes even more challenging when both the
27 robot's task execution and its architectural configuration need to be adapted. For instance, when a robot
28 runs out of battery while navigating, it must simultaneously adapt its architecture to a configuration that
29 consumes less energy and its task execution to include a recharge action and to navigate along paths that

are better suited to the new configuration. To address this challenge, robots can be designed as self-adaptive systems (SASS) with the ability to perform *task-and-architecture co-adaptation* (TACA) (Cámara et al., 2020), i.e., simultaneously adapt their task execution and software architecture dependently during runtime. This work focuses on proposing a systematic solution for enabling TACA that can be reused with different robotic systems.

A common approach to enable self-adaptation in software systems is to design them as two-layered systems containing a managing and managed subsystem (Weyns, 2020), where the managing subsystem monitors and reconfigures the managed subsystem, and the managed subsystem is responsible for the domain logic. This design facilitates the development and maintenance of the system by creating a clear separation between the adaptation and the domain logic. While several solutions have been proposed for solving either architectural (Alberts et al., 2025) or task adaptation in robotic systems (Carreno et al., 2021; Hamilton et al., 2022), there are some works that partially address TACA (Park et al., 2012; Lotz et al., 2013; Gherardi and Hochgeschwender, 2015; Valner et al., 2022), and there are few works that fully address TACA (Braberman et al., 2017; Cámara et al., 2020). More critically, to the best of our knowledge, the existing solutions for TACA require a significant and complex re-programming of the adaptation logic for each different use case, including the creation of multiple models based on different DSLs (Cámara et al., 2020) or implementing the managing subsystem itself (Braberman et al., 2017), hindering the adoption of SAS methods in robotics.

To address the limitations of SAS methods for TACA, this paper proposes to extend traditional robotics architectures with a novel knowledge-based managing subsystem for ROBOT Self-Adaptation (ROSA) that promotes reusability, composability, and extensibility. The main novelty of ROSA is its knowledge base (KB) which captures knowledge about the actions the robot can perform, the robot's architecture, the relationship between both, and their requirements to answer questions such as "What actions can the robot perform in situation X?" and "What is the best configuration available for each action in situation Y?", for example, "Can the robot perform an inspection action when the battery level is lower than 50%?" or "What is the best software configuration for the inspection action when the visibility is low?". This results in a reusable solution for TACA in which all application-specific aspects of the adaptation logic are captured in its KB.

In addition to a conceptual framework, this work provides a reference implementation of ROSA as an open-source framework that can be reused for research on self-adaptive robotic systems. ROSA is implemented as a ROS 2-based system Macenski et al. (2022), leveraging *TypeDB* (Dorn and Pribadi, 2023, 2024) for knowledge representation and reasoning, and *behavior trees* (BT) (Colledanchise and Ögren, 2018) as well as *PDDL-based planners* (Ghallab et al., 1998) for task decision-making.

The *feasibility* of using ROSA for runtime self-adaptation in robotic systems is demonstrated by applying it to the SUAVE exemplar (Silva et al., 2023), and its adaptation *performance* is evaluated in comparison to other approaches available in the exemplar. ROSA's *reusability* is demonstrated by using it to model the TACA scenarios described by Braberman et al. (2017) and Cámara et al. (2020). The *development effort* for using ROSA is evaluated by analyzing the number of elements contained in the knowledge models created to solve the aforementioned use cases and comparing it with the size of a BT-based approach used to solve SUAVE. ROSA's *development effort scalability* is demonstrated by showing how ROSA's knowledge model grows with the addition of extra adaptations in a hypothetical scenario.

In summary, the main contributions of this paper are:

- 72 1. a *modular architecture* for self-adaptive robotic systems that extends robotics architectures with a
73 managing subsystem and supports reusability, composability, and extensibility;
- 74 2. a *reusable knowledge model* to capture all application-specific aspects of the adaptation logic required
75 for TACA in self-adaptive robotic systems;
- 76 3. a reference *open-source implementation* of the framework that can be reused for self-adaptive systems
77 research; and
- 78 4. an *experimental evaluation* of ROSA based on simulated robotic self-adaptation scenarios.

79 The remainder of this paper is organized as follows. Section 2 describes the TACA use case used to
80 exemplify and evaluate this work. Section 3 presents related works. Section 4 describes how this work
81 proposes to extend robotics architectures with ROSA. Section 5 details ROSA's KB. Section 6 describes
82 the proposed reference implementation of ROSA. Section 7 showcases ROSA's evaluation. Section 8
83 concludes this work and presents future research directions.

2 RUNNING EXAMPLE

84 Throughout this paper, the SUAVE exemplar (Silva et al., 2023) is used as an example to ease the
85 understanding of the proposed solution, and later, it is used to evaluate ROSA.

86 SUAVE consists of an Autonomous Underwater Vehicle (AUV) used for underwater pipeline inspection.
87 The AUV's mission consists of performing the following actions in sequence: (A1) *searching for the*
88 *pipeline* and (A2) *simultaneously following and inspecting the pipeline*. When performing its mission, the
89 AUV is subject to two uncertainties: (U1) *thruster failures*, and (U2) *changes in water visibility*. These
90 uncertainties are triggers for parameter and structural adaptation. When U1 occurs while performing A1 or
91 A2, the AUV activates a functionality to recover its thrusters. When U2 happens while performing A1, the
92 AUV adapts its search altitude.

93 To demonstrate TACA, this work extends SUAVE with an (A3) *recharge battery* action and a (U3) *battery*
94 *level* uncertainty. With these extensions, the AUV's battery level can suddenly drop to a critical level,
95 requiring the AUV to abort the action it is performing (A1 or A2) and perform A3. In this situation, the
96 AUV needs to perform TACA by adapting its task execution and architecture to perform A3. To better
97 evaluate ROSA by serving as a baseline for comparison, this work extends the SUAVE exemplar with a
98 managing subsystem where the adaptation logic is implemented with BTs, and the AUV's architectural
99 variants as well as the architectural adaptation execution are realized with System Modes (Nordmann et al.,
100 2021)¹. Furthermore, this work introduces a new *reaction time* metric that represents the time a managing
101 system takes to react to uncertainties and adapt the managed subsystem.

3 RELATED WORK

102 This work combines principles from self-adaptive systems and knowledge representation and reasoning to
103 design a reusable framework for developing adaptive robotics architectures. Section 3.1 analyzes existing
104 robotics architectures and describes the architectural patterns from robotics architectures adopted in this
105 work. Section 3.2 reviews related research on self-adaptive robotic systems that leverage knowledge
106 representation techniques to promote reusability, as well as studies that consider the relationship between

¹ The SUAVE exemplar is already configured to use System Modes

task execution and architectural adaptation. Additionally, it discusses how these works influenced the design of the proposed framework and highlights its distinctions from existing approaches.

3.1 Robotics Architectures

Numerous approaches have been proposed for programming and designing autonomous robot architectures (Kortenkamp et al., 2016). In recent years, two main trends have emerged: component-based frameworks and middlewares—among which ROS Macenski et al. (2022) stands out due to its widespread adoption in academia and industry—and layered architectures (Barnett et al., 2022). Barnett et al. (2022) reviewed 21 robotics architectures and concluded that most architectures follow a layered pattern, and even those that do not can still have their elements mapped onto a layered architectural structure. Furthermore, they found that all architectures include a bottom functional layer responsible for interacting with the robot's hardware, an upper task decision layer—whose responsibilities vary across architectures—and an arbitrary number of intermediate layers. This work aims to design a reusable solution for TACA that can be integrated into robotics architectures adhering to these architectural patterns. To achieve this, the proposed solution establishes a clear separation between architectural management and task logic, organizing them into distinct layers, or subsystems, as commonly referred to in the self-adaptive systems community.

The LAAS architecture (Alami et al., 1998) is an example of a three-layered architecture consisting of a functional layer, an executive, and a decision layer. The functional layer contains the robot's control and perception algorithms. The executive layer receives a task plan from the decision layer and selects functions from the functional layer to realize each action in the task plan. The decision layer includes a planner that generates task plans and a supervisor responsible for monitoring plan execution and triggering replanning when necessary. More recent examples of layered robot architectures include AEROSTACK (Sanchez-Lopez et al., 2016), designed for aerial drone swarms, and SERA (Garcia et al., 2018), which is tailored for decentralized and collaborative robots. These architectures build on the layered model but focus on providing domain-specific solutions.

Cognitive architectures (Kotseruba et al., 2018), such as CRAM (Beetz et al., 2010; Kazhoyan et al., 2021), focus on generating intelligent and flexible behavior by integrating cognitive capabilities such as planning, perception, or reasoning. However, being integral solutions, these works provide a blueprint for the complete robot control system and are not intended for reuse and integration with other methods, thus making it difficult to adapt and customize for specific applications.

3.2 Self-adaptive robotic systems

Despite advances in self-adaptive robotic systems, fully addressing task and architectural co-adaptation (TACA) with reusable and scalable solutions remains an open challenge. While some studies explore the relationship between task execution and architectural adaptation, only a few explicitly address TACA—and those that do face limitations in reusability and practical applicability in robotics. This work aims to bridge this gap by introducing a knowledge-based framework that can capture the necessary knowledge to solve TACA across different use cases, can be directly applied to robotic systems, and supports modular modifications for incorporating different adaptation strategies.

Alberts et al. (2025) recently conducted a systematic mapping study² on “robotics software architecture-based self-adaptive systems” (RSASSs), identifying 37 primary studies on RSASSs published since 2011. Among these, Alberts et al. (2025) identified that four studies (Park et al., 2012; Lotz et al., 2013; Gherardi

² Alberts et al. (2025) do not claim that the mapping study is a complete overview of the literature but rather a characterization of the field

Table 1. Related frameworks for robot self-adaptation. Where A-t-T means “Architectural state triggers task adaptation”, and T-t-A means “Task triggers architectural adaptation”.

Approach	Architectural adaptation		TACA		Reusability		Robotics Middleware	Applied to robot
	Parameter	Structural	A-t-T	T-t-A	Conceptual	Software Available		
ICE (Niemczyk et al., 2017)	No	Yes	No	No	No	No	None	No
Hochgeschwender et al. (2016)	No	Yes	No	No	No	No	None	Real Robot
Metacontrol (Bozhinoski et al., 2022)	Yes	Yes	No	No	Yes	Yes	ROS 1&2	Real Robot
SHAGE (Park et al., 2012)	No	Yes	No	Yes	Partially	No	None	No
Lotz et al. (2013)	No	Yes	No	Yes	No	No	None	No
RRA (Gherardi and Hochgeschwender, 2015)	Yes	Yes	No	Partially	Yes	Yes	ROS 1	Simulated
TeMoto (Valner et al., 2022)	Yes	Yes	No	Yes	Partially	Yes	ROS 1	Real robot
MORPH (Braberman et al., 2017)	Yes	Yes	Yes	Yes	No	No	None	No
Cámara et al. (2020)	Yes	Yes	Yes	Yes	Partially	No	ROS 1	Simulated
ROSA	Yes	Yes	Yes	Yes	Yes	Yes	ROS 2	Simulated

and Hochgeschwender, 2015; Cámara et al., 2020) consider, to varying degrees, the relationship between the tasks a robot performs and architectural adaptation. A non-systematic snowballing of the primary studies identified by Alberts et al. (2025) revealed two additional studies (Braberman et al., 2017; Valner et al., 2022) that also explore this relationship.

In the context of knowledge-based methods, Alberts et al. (2025) identified six studies (Park et al., 2012; Niemczyk and Geihs, 2015; Hochgeschwender et al., 2016; Niemczyk et al., 2017; Bozhinoski and Wijkhuizen, 2021; Silva et al., 2023) that use knowledge representation techniques to capture knowledge required for the adaptation logic. Among these, (Bozhinoski and Wijkhuizen, 2021; Silva et al., 2023) do not propose solutions for RSASSs but instead demonstrate the application of Metacontrol (Hernández et al., 2018; Bozhinoski et al., 2022) in different robotics use cases. While these methods do not address TACA, they provide valuable insights for designing knowledge-based approaches to self-adaptation.

Park et al. (2012) introduced the SHAGE framework for task-based and resource-aware architecture adaptation in robotic systems. SHAGE partially solves TACA, as it can adapt the robot’s architecture at runtime to specifically realize each action in its task plan when it needs to be performed. However, SHAGE does not support task execution adaptation based on the robot’s architectural state. SHAGE promotes reusability by leveraging architectural models and knowledge captured with an ontology to reason about adaptation at runtime. However, to the best of our knowledge, there is no implementation of the SHAGE framework that works with common robotic frameworks. Thus, it is not possible to directly reuse SHAGE.

Lotz et al. (2013) proposed a method to model operational and quality variability using two distinct domain-specific languages (DSLs) models. Their work provides a high-level discussion on how these models could be used at runtime to enable architectural adaptation based on the actions executed by the robot. An interesting aspect of their approach is the clear separation between functional and non-functional requirements: one model captures the task deliberation logic, functional requirements, and their variation points, while the other focuses on non-functional requirements and their possible variations. While this separation of concerns simplifies the modeling process, combining task deliberation with functional requirements reduces reusability. Any change in the task deliberation logic directly impacts the modeling of functional requirements, making the approach less flexible. Additionally, they do not provide sufficient details on how these models are used at runtime, nor do they present an evaluation to demonstrate the feasibility of their approach.

Gherardi and Hochgeschwender (2015) proposed RRA as a model-based approach for structural, parameter, and connection adaptation in robotic systems. Their method employs six distinct models to capture all the knowledge required for adaptation. These models represent the robot’s architecture and

its variability, its functionalities and their variability, the mapping between functions and architecture, the required interfaces (i.e., inputs, outputs, and data types) for the adaptation logic, and the adaptation logic itself. RRA models the dependencies between the tasks a robot can perform and the architectural configurations needed to accomplish them. This is achieved by decomposing each task into multiple functionalities and capturing the available architectural variants for realizing each function. At deployment time, the robot's operator selects a task, and RRA manages only the functionalities required for that task. Although RRA considers the relationship between tasks and architecture to some extent, it cannot be classified as TACA, as this dependence is only accounted for at deployment time. Architectural adaptation occurs based on the selected task rather than dynamically at runtime in response to the individual actions the robot needs to perform. Moreover, RRA does not adapt the task execution based on the robot's architectural state.

Valner et al. (2022) proposed the TeMoto as a general solution for robotic systems' dynamic task and resource management. TeMoto partially solves TACA, as it can adapt the robot's architecture to realize the actions being performed by the robot. TeMoto does not completely fulfill TACA as it cannot adapt the task execution given the robot's architectural state. TeMoto provides reusable mechanisms for resource management, but reusability is limited since the adaptation logic must be implemented for all managed resources, and the knowledge about the dependencies between actions and architecture are programmatically included in the actions' code.

Braberman et al. (2017) proposed MORPH as a reference architecture to enable TACA. They showcased on a conceptual level how MORPH can be applied to enable TACA in an unmanned aerial vehicle (UAV) use case. However, since MORPH is only demonstrated at a conceptual level, it is hard to evaluate the feasibility of applying MORPH to robotic systems at runtime. To the best of our knowledge, there is no framework implementing the complete MORPH architecture. Therefore, it is not possible to directly reuse MORPH.

In the context of TACA, Cámara et al. (2020) developed a method for finding *optimal* task and reconfiguration plans for an autonomous ground vehicle (AGV) navigating in a graph-like environment. To enable optimal planning within reasonable time limits, their method first reduces the search space by finding all possible reconfiguration plans and then computing the shortest N paths the robot can take to reach its goal. Then, it uses this information along with task-specific models that capture mission quality attributes (e.g., energy consumption, collision probabilities) and a preferred utility to apply model checking and determine an optimal reconfiguration plan for each path. Finally, an optimization function selects the best plan based on a predefined utility function (e.g., minimizing energy consumption, time, or collision probability). Although their approach reduces the planning search space to improve planning time, their experiments show that solving the navigation use case still takes an average of 15.1 seconds, an impractical duration for robots that frequently need to replan at runtime to handle uncertainties. Additionally, while the approach is model-based, it relies on task-specific model transformations (e.g., converting the map or battery model into PRISM model snippets), which require dedicated implementations for different tasks. This limits the reusability of their approach for different types of tasks, as it requires a considerable amount of development effort.

Hochgeschwender et al. (2016) argue that robots should have access to and exploit software-related knowledge about how they were engineered to support runtime adaptation. They demonstrate how labeled property graphs (LPGs) can be used to persistently store and compose different domain models specified with domain-specific languages (DSL) to enable runtime architectural adaptation. Although their approach is interesting, its reusability is limited as it does not define a knowledge model that can be reused for other

applications: for each different use case and DSL the roboticist is responsible for creating a translation from the DSL to the corresponding LPG.

Niemczyk and Geihs (2015); Niemczyk et al. (2017) propose ICE as a method for adapting the information processing subsystem in multi-robot systems, specifically by adapting the connections between system components. Their approach uses an ontology to define each component's required inputs and outputs, along with quality-of-service information for each connection. This ontology is then translated into answer set programming (ASP), and an ASP solver determines an optimal configuration. While they conceptually demonstrate how their method could be applied to robots, they do not demonstrate it with a robotic system. Moreover, their approach focuses solely on structural and connection adaptation to maintain the functionality of the information processing subsystem, without considering other subsystems of the robotic system.

Hernández et al. (2018); Bozhinoski et al. (2022) proposed Metacontrol as a knowledge-based solution for parameter and structural adaptation. Metacontrol leverages the TOMASys (Hernández et al., 2018) ontology to capture the knowledge required for the adaptation logic and to reason at runtime to decide when and how the system should adapt. Similar to RRA, Metacontrol decomposes the system into functionalities, using TOMASys to represent the robot's functionalities, the architectural variants that implement each functionality, and the non-functional requirements associated with these variants. However, Metacontrol cannot perform TACA as TOMASys does not capture the relationship between the system functionalities and the robot's actions.

In conclusion, existing works that fully address TACA (Braberman et al., 2017; Cámara et al., 2020) face limitations in reusability and practical applicability in robotics. The authors of MORPH (Braberman et al., 2017) note that no complete system has been developed, and they have not demonstrated its feasibility. While the approach proposed by Cámara et al. (2020) provides the most complete solution to TACA in the literature, it also faces reusability limitations by relying on multiple distinct domain-specific language (DSL) models and requiring task-specific implementations for model transformations. To address these limitations, this work proposes capturing all the knowledge required for adaptation logic in a single knowledge model. This approach requires only one model—conforming to the proposed knowledge model—to be designed for each application. Additionally, an open-source reference implementation of ROSA is provided, enabling researchers to reuse, extend, and build upon the proposed solution.

On the other hand, previous knowledge-based methods for RSASS (Park et al., 2012; Niemczyk and Geihs, 2015; Niemczyk et al., 2017; Hochgeschwender et al., 2016; Hernández et al., 2018; Bozhinoski et al., 2022) do not capture all knowledge required to enable TACA. They either lack the ability to capture the relationship between the robot's actions and architecture (Hochgeschwender et al., 2016; Hernández et al., 2018; Bozhinoski et al., 2022), or the knowledge required to decide when and how the robotic system should adapt (Park et al., 2012). Although these works do not capture all the knowledge required for TACA, they are able to capture, to varying degrees, knowledge that supports RSASSs. Thus, this work takes inspiration from them to design ROSA's knowledge model while addressing their limitations. More concretely, ROSA's knowledge model is designed to capture the relationship between the robot's actions and architecture and the knowledge required to decide when and how TACA should be performed.

4 ARCHITECTURE

To enable TACA, this paper proposes to extend traditional robotics architectures with ROSA, using it as a managing subsystem for the robotic subsystem (see Figure 1). This section first describes the assumptions

made about the robotics architecture and the requirements to use it alongside ROSA, and then it details ROSA's architecture.

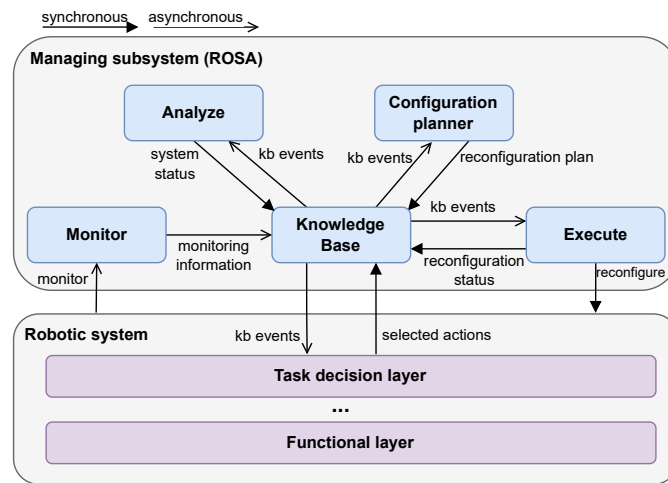


Figure 1. The upper layer depicts ROSA's architecture and the bottom layer depicts the robotic system.

4.1 Robotics architecture

This work assumes that the robotics architecture is layered, containing a bottom functional layer, an upper task decision layer, and an arbitrary number of layers in between, as common in robotics architectures (Barnett et al., 2022). The functional layer is responsible for interacting with the robots' sensors and actuators, and the task decision layer is responsible for task planning and execution³. To enable TACA with ROSA, the task decision layer shall use the knowledge contained in ROSA's KB to decide which actions to perform, and it must update the KB with the actions selected to be performed to enable ROSA to configure the robot's architecture accordingly. To enable architectural adaptation, the robotic architecture must be component-based, its components must be able to be activated and deactivated at runtime, and its components' parameters must be able to be adapted at runtime.

4.2 ROSA architecture

ROSA's architecture adheres to the MAPE-K loop (Kephart and Chess, 2003). It *monitors* the managed subsystem, *analyzes* whether adaptation is required, when needed, *plans* how the managed subsystem should be reconfigured, and *executes* the selected reconfigurations. All these steps interact with a central KB.

To promote reusability, composability, and extensibility, the architecture is designed with the following premises: (1) *all* knowledge required for the adaptation logic is captured in the central KB, (2) there is no inter-component communication between the MAPE components (Weyns et al., 2013), (3) the MAPE components insert and read data from or to the KB via standardized interfaces, and (4) there is no explicit coordination between the MAPE-K components. Premise 1 promotes reusability by only requiring the modeling of the relevant knowledge for applying ROSA to different applications in a single model. Premises 1 to 4 promote composability and extensibility by allowing the MAPE components to be stateless and self-contained.

³ Some architecture have distinct layers for handling task planning and execution, in the context of this work, they can be considered as sub-layers of the task decision layer.

5 KNOWLEDGE BASE

To fulfill the architectural premise that all knowledge required for the adaption logic should be captured in a central KB, this work proposes a KB component composed of the knowledge model depicted in Figure 2 and the set of rules depicted in Figure 3, described in Section 5.1 and Section 5.2 respectively.

The knowledge model is presented as a conceptual data model (CDM) conforming to a particular case of the enhanced entity-relationship (EER)Thalheim (1993, 2000) model, an extension of the entity-relationship model Chen (1976) that accounts for subclassing and higher-order relationships, i.e., relations between relationships. The EER model captures information as entities, relationships, and attributes. An entity is a “‘thing’ which can be distinctly identified” Chen (1976), a relationship is an association among entities or relationships, and an attribute represents a property of an entity or relationship Thalheim (1993, 2000). In addition, entities and relationships play a role in the relationships they are part of, which is identified by the label on the arrows in Figure 2. This CDM was selected since it supports n-ary relationships, many-to-many relationships, higher-order relationships, and attributes for both entities and relationships. Section 6.1 details why these representation capabilities are relevant to the proposed model. The rules are also described at a conceptual level as decision diagrams. Section 6.2 presents the details on how the knowledge model and rules can be implemented and executed and runtime.

5.1 Knowledge model

To enable adaptation, the knowledge model captures *what* can be adapted with the architectural knowledge depicted in Figure 2a; *why* to adapt and *how* to select an adaptation with the adaptation heuristics knowledge depicted in Figure 2b; and *how* to execute an adaptation with the reconfiguration plan knowledge depicted in Figure 2c. Tables 2-4 define each element in the model alongside examples based on SUAVE to ease its understanding.

5.1.1 Architectural knowledge

The architectural knowledge (see Table 2) captures what actions the robot can accomplish, the set of functionalities the robot needs to realize an action, the set of components required to realize a functionality, and the possible parameters for a component.

The architectural knowledge enables parameter adaptation by capturing each parameter configuration of a component with a distinct `component configuration` relationship, relating one `Component` to a set of `Component Parameters`. It enables structural adaptation by capturing the different possibilities for solving a system functionality as distinct `function design` relationships which relate a `Function` to a set of `Components`. It enables TACA by indirectly capturing the dependencies between the robot’s actions and architecture with the `functional requirement` relationship which relates an `Action` to the `Functions` it requires.

At runtime, ROSA performs parameter adaptation by switching the selected `component configurations`. It performs structural adaptation by changing the selected `function designs` and consequently the active `Components`. The task decision layer in combination with ROSA performs TACA by selecting suitable `function designs` and `component configurations` for each `Action` the robot needs to perform, and with the task decision layer selecting different `Actions` to perform according to the feasible configurations.

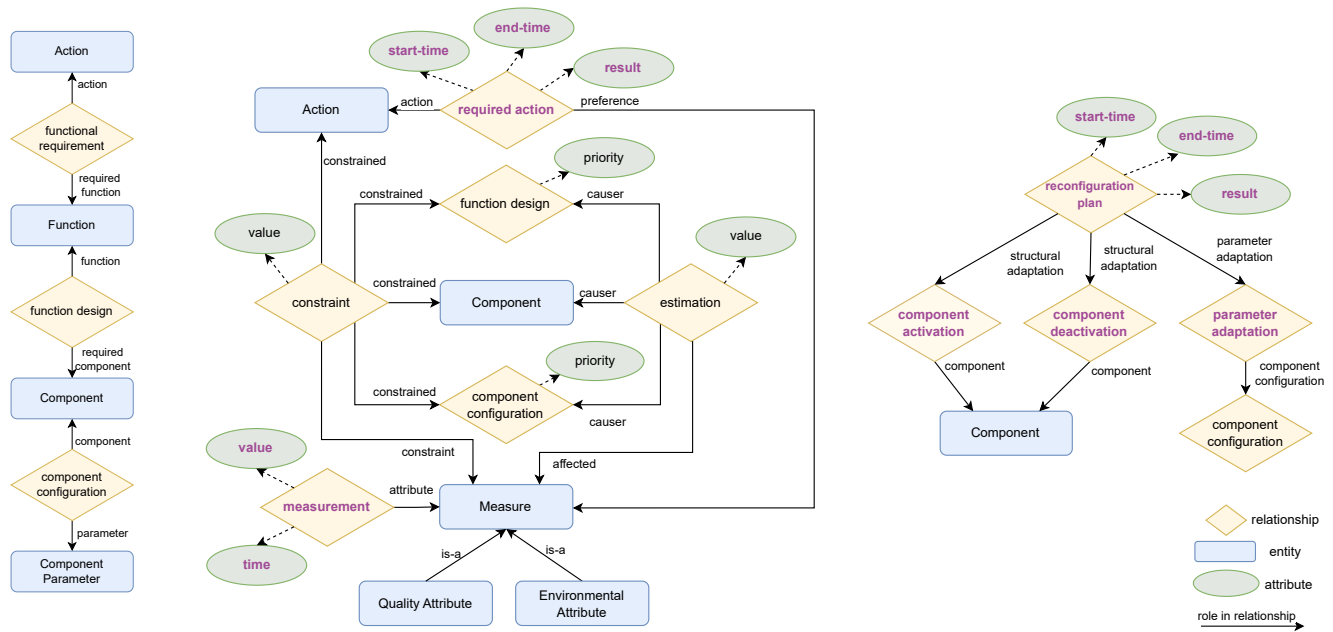


Figure 2a. Architectural **Figure 2b.** Adaptation heuristic

Figure 2c. Reconfiguration plan

Figure 2. ROSA's knowledge model. Architectural knowledge is on the left. Adaptation heuristic knowledge in the center. Reconfiguration Plan knowledge on the right. The labels on the arrows represent which role an entity or relationship plays in a relationship. Instances of the entities, relationships, and attributes with purple font are created at runtime. Instances of the other entities, relationships, and attributes are defined at design time.

5.1.2 Adaptation heuristic knowledge

This work considers that the robotic system might need to adapt due to changes in the environment, changes in the system's quality attributes (QAs) (Board, 2023), component failures, and changes in the robot's selected actions.

The adaptation heuristic knowledge enables adaptation due to changes in the environment or the system's QAs with the `constraint` relationship by capturing constraints on the selection of Actions, Components, function designs, or component configurations in terms of measured values of Measures. It enables adaptation due to component failures by capturing a Component's status as an attribute and adaptation due to changes in the robot's task execution by capturing what Actions need to be performed with the `required action` relationship. At runtime, adaptation is triggered when a measurement violates a constraint, when a component has a failure status, or when required actions change.

To capture the decision criteria on how to select an adaptation, the `priority` attribute can be used to express the order of priority for selecting each function design or component configuration. For more complex criteria, the `estimation` relationship can be used to capture the estimated impact of selecting a function design, Component, or component configuration on the measured values of Measures. Furthermore, a `required action` can relate to a Measure to indicate the preferred estimation when selecting a configuration for that action. At runtime, the configuration planner component exploits this knowledge to decide which configuration to select.

Table 2. The elements of the architectural knowledge. Instances of the elements with **purple font** are created at runtime. Instances of the other elements are defined at design time. The ‘@key’ expression after an attribute indicates that the attribute is used as a unique identifier. E stands for entity and R stands for relationship.

Name	Definition	Example	Attributes
E	Action	An AUV can perform the <i>Action search pipeline</i> , and <i>inspect pipeline</i>	name:String @key status:String is-required:Bool
	Function	An AUV can have several Functions, like <i>generate search path</i> and <i>generate path to follow pipeline</i> . The <i>generate search path</i> function can be considered a transformation of the AUV’s current position (input) to a goal waypoint (output)	name:String @key always-improve:Bool status:String is-required:Bool
	Component	Hardware and software parts that compose the system	name:String @key always-improve:Bool status:String is-required:Bool is-active:Bool pid:Integer
	Component Parameter	A specific parameter configuration of a Component	key:String value:String
R	functional requirement	Represents which Functions a certain Action requires to be performed	N/A
	function design	Represents a solution for a Function as a set of Components (Hernández et al., 2018)	name:String @key priority:Integer status:String is-selected:Bool
	component configuration	Represents a possible configuration of a Component as a set of Component Parameters	name:String @key priority:Integer status:String

5.1.3 Reconfiguration plan knowledge

The reconfiguration plan knowledge represents which component parameters must be updated (i.e., how to execute parameter adaptation) and which components must be activated and deactivated (i.e., how to execute structural adaptation). The execute component exploits this knowledge at runtime to reconfigure the managed subsystem.

Table 3. The elements of the adaptation heuristic knowledge. Instances of the elements with purple font are created at runtime. Instances of the other elements are defined at design time. The '@key' expression after an attribute indicates that the attribute is used as a unique identifier. E stands for entity and R stands for relationship.

Name	Definition	Example	Attributes
E	Measure	An AUV can have the Measures <i>battery level</i> or <i>water visibility</i>	name:String @key
	Quality Attribute	An AUV can have the Quality Attributes <i>battery level</i> , <i>battery consumption</i> , and <i>safety level</i>	name:String @key
	Environmental Attribute	An underwater environment can have <i>water visibility</i> as an Environmental Attribute	name:String @key
R	required action	The <i>inspect pipeline</i> Action can be required to be performed at runtime, leading ROSA to configure the AUV appropriately to carry out the inspection action	start-time:Datetime end-time:Datetime result:String
	measurement	The <i>battery level</i> Quality Attributes can have a measurement of 0.5	value:Double time:Datetime
	constraint	If the <i>water visibility</i> environmental attribute is low, the <i>high altitude</i> configuration for the <i>generate spiral search path node</i> cannot be selected	operator:String value:Double status:String
	estimation	A lamp Component is expected to positively impact the <i>water visibility</i> when turned on	value:Double type:String

5.2 Rules

To enable ROSA to reason about when the managed subsystem should be adapted and what type of adaptation is needed, the KB contains a set of generic rules that define how the status of the system can be inferred based on monitored information, e.g., how to infer if a constraint is violated and how to propagate a constraint violation. A change in status only occurs when measurements are updated or when a component fails. The complete status inference rules are depicted as decision diagrams in Figure 3.

The status of the required Components, Functions, and Actions indicate when and what type of adaptation is needed. When a Component is “unsolved” or in “configuration error” (see Figure 3c), parameter adaptation is required, i.e., a new component configuration needs to be selected. When a required Function is “unsolved” or in “configuration error” (see Figure 3e), structural adaptation is required, i.e., a new function design needs to be selected. When a required Action is “unfeasible” (see Figure 3f), TACA is needed, i.e., the task decision layer must choose a new action to perform, consequently triggering architectural adaptation.

Table 4. The elements of the reconfiguration plan knowledge. Instances of the elements with purple font are created at runtime. Instances of the other elements are defined at design time. R stands for relationship.

Name	Definition	Example	Attributes
R reconfiguration plan	Represents the reconfiguration plan generated in the plan step	When the AUV completes the <i>search pipeline</i> action and starts the <i>inspect pipeline</i> action, the reconfiguration plan consists of deactivating the <i>generate spiral search path node</i> and activating the <i>follow pipeline node</i>	start-time:Datetime end-time:Datetime result:String
component activation	Represents the components that should be activated	When the AUV starts the <i>inspect pipeline</i> action, the component activation relates to the <i>follow pipeline node</i>	N/A
component deactivation	Represents the components that should be deactivated	When the AUV completes the <i>search pipeline</i> action, the component deactivation relates to the <i>generate spiral search path node</i>	N/A
parameter adaptation	Represents the component parameters that should be updated	When the water visibility changes, the parameter adaptation consists of a parameter adaptation of the component configuration of the <i>generate spiral search path node</i>	N/A

6 ARCHITECTURE REALIZATION

This section details the proposed reference implementation for ROSA. First, the representation requirements imposed by the proposed knowledge model are analyzed to select a suitable knowledge representation technique. Afterward, the proposed implementation is explained.

6.1 Representation requirements

The proposed knowledge model was presented as a conceptual data model (CDM) in a non-machine-readable format. To transform the knowledge model into a machine-readable format while maintaining its semantics and structure, its representation requirements must be identified and used to select a suitable technology to implement it. The representation requirements imposed by the proposed knowledge model are:

1. n-ary relationships: the knowledge model contains relationships with different arity⁴, e.g., the *constraint* relationship has arity 5, and the *measurement* relationship has arity 1;
2. many-to-many relationships: the knowledge model contains relationships with different cardinalities⁵, e.g., *function design* is a 1 *Function*-to-many *Components* relationship;
3. higher-order relationships: some relationships relate relationships to relationships, e.g., the *constraint* relationship;
4. attributes for entities and relationships: both entities and relationships have attributes, e.g., the *Action* entity and the *constraint* relationship.

These requirements limit which technology can be used. For example, technologies using graph-based or descriptive logic-based knowledge representation techniques (e.g., OWL (Antoniou and van Harmelen,

⁴ the number of distinct elements that can be part of a relationship

⁵ the number of element instances that can be part of a relationship

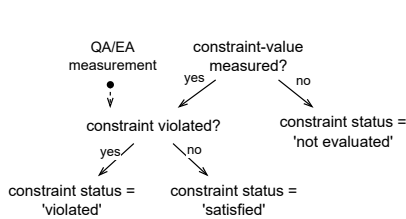


Figure 3a. Constraint status

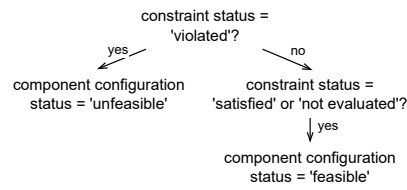


Figure 3b. Component configuration status

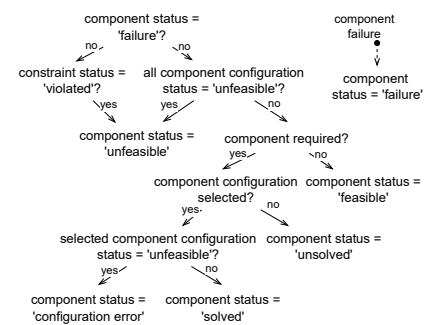


Figure 3c. Component status

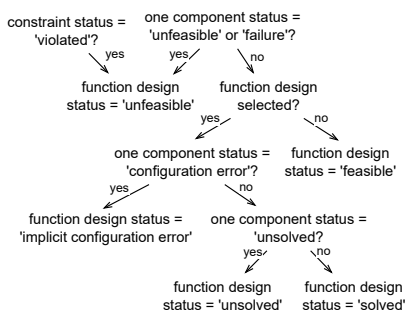


Figure 3d. Function design status

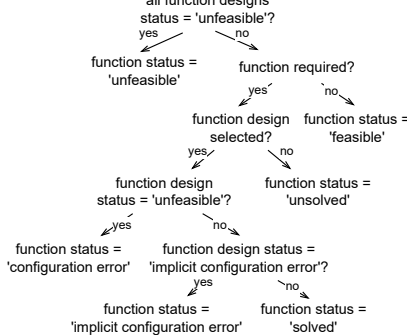


Figure 3e. Function status

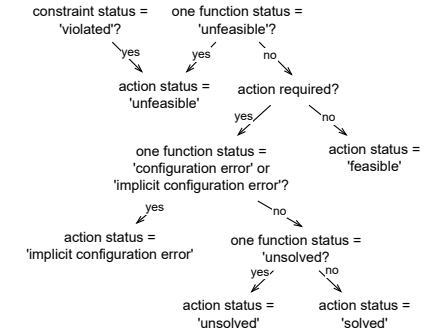


Figure 3f. Action status

Figure 3. Rules used to infer the status of the elements in the knowledge model, presented as decision diagrams

2004)) do not satisfy the representation requirements above apart from allowing attributes for entities (part of Requirement 4). Although it is possible to transform the proposed knowledge model into one that can be represented with graph-based or description logic-based approaches by reifying the model (Olivé, 2007), each reification applied to the original model can be considered as an introduction of a semantic disparity between the CDM and the machine-readable model, making it harder to understand and reuse it. Thus, this work does not consider applying reification.

A technology that satisfies all representation requirements is TypeDB (Dorn and Pribadi, 2023, 2024). TypeDB is a polymorphic database based on type theory that implements the polymorphic entity-relation-attribute (PERA) Dorn and Pribadi (2024) data model. The PERA model subsumes the CDM used as the meta-model for the proposed ROSA model, allowing it to be implemented without modifications. Furthermore, TypeDB has a reasoning system that is able to reason over rules of the form *antecedent* $\Rightarrow$ *consequent* to infer new facts at query time. Where *antecedent* represents a precondition for inferring the *consequent* and is expressed as a first-order logic expression combining elements from the model (i.e., entities, relationships, and individuals), and the *consequent* is a single new fact inferred when the *antecedent* holds true. The ROSA model rules presented in Figure 3 can be implemented with TypeDB without modifications. For these reasons, this work uses TypeDB to implement the proposed knowledge model and rules.

6.2 Implementation

ROSA is implemented as a ROS 2-based system, where the MAPE-K components (depicted in Figure 1) are realized as ROS nodes, and interfaces are implemented using ROS services or topics. The proposed ROSA implementation uses ROS (Robot Operating System) as its robotics framework since ROS is the current de facto standard robotics framework, and it has been designed, among other things, to promote software reusability in the robotics ecosystem Macenski et al. (2022). In this implementation, ROS handles the communication between system components, schedules callbacks for incoming messages and events, and manages the lifecycle of ROS nodes. The full ROSA implementation is available at <https://github.com/kas-lab/rosa>⁶.

6.2.1 Knowledge base and analyze

The KB component consists of the TypeDB implementation of the proposed knowledge model and inference rules, the ROS interfaces for communicating with the MAPE components, and the logic to manage ROSA's knowledge which is stored in a TypeDB database. In this reference ROSA implementation, TypeDB's reasoner fulfills the role of the analyze component, executing ROSA's inference rules (Figure 3) to infer new data when the KB is queried. Thus, since TypeDB's reasoner is part of TypeDB, there is no separate analyze component.

Knowledge model and rules: To exemplify how the knowledge model is implemented, Listing 1 depicts how the `functional-requirement` relationship and the `Action` entity are defined with TypeQL (TypeDB's query language). Line 1 defines the `functional-requirement` relationship, and lines 2-3 define that it can relate elements that play the role of *actions* and *required-functions*. Lines 5-7 define the `Action` entity and that it has the attributes 'action-name' (its unique identifier) and 'action-status'. Lines 8-9 define that it can play the *action* role in a `functional-requirement` relationship and the *constrained* role in a `constraint` relationship.

```
functional-requirement sub relation,
  relates action,
  relates required-function;

Action sub entity,
  owns action-name @key,
  owns action-status,
  plays functional-requirement:action,
  plays constraint:constrained;
```

Listing 1. TypeQL query to define `functional-requirement` and `Action`

Listing 2 exemplifies how the inference rules are implemented with TypeQL. The `component-status-configuration-error` rule defines that a `Component` has a 'configuration error' status when it is required, it does not have an 'unfeasible' or 'failure' status, and it is in a `component-configuration` relationship that is selected and has an 'unfeasible' status (see Figure 3c).

```
rule component-status-configuration-error:
  when {
    $c isa Component, has is-required true;
  }
  not {
```

⁶

In addition to ROSA, this work provides a generic ROS package to integrate ROS 2 with TypeDB: https://github.com/kas-lab/ros_typedb

```

4435  $c has status $c_status;
4447  $c_status like 'unfeasible|failure';
4458  };
4469  (component: $c) isa component-configuration,
4470  has is-selected true,
4481  has status 'unfeasible';
4492  } then {
4503  $c has status 'configuration error';
4514  };

```

Listing 2. TypeQL rule to infer whether a component is in ‘configuration error’

453 *Interfaces:* The KB component abstracts the details of interacting with TypeDB with the ROS interfaces
 454 it implements, enabling the MAPE components to read and write knowledge via the interfaces described
 455 in Table 5. When the MAPE components request or send data to the KB component via these interfaces,
 456 the KB component queries the TypeDB database to retrieve or write knowledge. For example, when the
 457 task decision layer calls the service `/action/selectable` to retrieve the name of the selectable `Actions` (i.e.,
 458 actions that do not have an ‘unfeasible’ status), the KB component performs the TypeQL query depicted
 459 in Listing 3 to retrieve the name (unique identifiers) of the selectable `Actions`. When data is written in
 460 the KB, the KB component publishes a message in the `/events` topic specifying which type of data was
 461 written, i.e., ‘monitoring data’, ‘action update’, ‘reconfiguration plan’. Additionally, the KB component
 462 provides the `/query` service, which can be used to perform *any* TypeDB query to the database. It is not used
 463 in ROSA’s runtime workflow, but it enables users to perform custom queries, for example, to retrieve all
 464 reconfiguration plans that were executed.

Table 5. ROS interfaces

Topics		
Publisher	Subscriber	Name
KB	Configuration Planner	~/events
	Execute Task Decision Layer	
Monitor nodes	KB	/diagnostics
Services		
Server	Client	Name
KB	Configuration Planner	~/function/adaptable
		~/function_designs/selectable
		~/function_designs/priority
		~/component/adaptable
		~/component_configuration/selectable
		~/component_configuration/priority
	Execute	~/select_configuration
		~/reconfiguration_plan/get_latest
		~/reconfiguration_plan/result/set
		~/component/active/set
	Task Decision Layer	~/component_parameters/get
		~/action/selectable
	User	~/action/request
		~/query

```

1 match $a isa Action, has name $name;
2   not {$a has status 'unfeasible'};
3   fetch $name;

```

Listing 3. TypeDB query to fetch selectable actions' names

465 6.2.2 Monitor

466 ROSA's implementation does not provide generic monitor nodes. They should be implemented as needed
 467 for each application with the requirement that they publish the monitored information in the */diagnostics*
 468 topic⁷ with the standard ROS *DiagnosticArray* message format. When a monitor node sends measurement
 469 updates to the KB, the message field in the *DiagnosticStatus* message needs to be set to 'QA measurement'
 470 or 'EA measurement', and when sending component status updates (e.g., that the component is in failure),
 471 the message field must be set to 'Component status'. When the KB receives monitoring data, it sends an
 472 event message in the '/events' topic to inform that monitoring data was written in the KB.

473 6.2.3 Configuration planner

474 The configuration planner component selects the configurations (i.e., function designs or
 475 component configuration) with the highest priority. When the configuration planner receives an
 476 event message indicating that monitoring data was written in the KB or that there was an update in the
 477 required actions, it calls the services '/function/adaptable' and '/component/adaptable' to check which
 478 Functions and Components must be adapted. Then, it calls the services '/function/selectable' and
 479 '/component/selectable' to check which function designs and component configurations are
 480 available for the Functions and Components that need to be adapted. Finally, the configuration planner
 481 selects the function designs and component configurations with the highest priority and
 482 informs the KB about the newly selected configuration by calling the service '/select_configuration'. When
 483 this service is called, the KB component checks the current state of the robot, creates a *reconfiguration*
 484 plan to bring the robot to the goal configuration, and sends an event message in the '/events' topic to
 485 inform that there is a new reconfiguration plan available.

486 6.2.4 Execute

487 In ROS-based systems, software components are realized either as ROS nodes or as a particular type
 488 of ROS nodes called *lifecycle* nodes. The difference between both is that the latter can be set to different
 489 states at runtime, such as *active* and *inactive*, and the former cannot. To enable ROSA to leverage ROS 2
 490 mechanisms to adapt the system, the knowledge model was extended to capture knowledge about ROS 2
 491 components as depicted in Figure 4. The execute component performs structural adaptation by starting or
 492 killing ROS nodes or switching the state of lifecycle nodes to active or inactive, and it performs parameter
 493 adaptation by calling the ROS's parameter API to change the ROS nodes' parameters at runtime.

494 When the execute component receives an event message indicating that a new reconfiguration plan was
 495 added to the KB, it calls the service '/reconfiguration_plan/get_latest' to get the latest *reconfiguration*
 496 plan. Then, it adapts the robot's architecture according to the reconfiguration plan. Finally, it calls the
 497 services '/reconfiguration_plan/result/set' and '/component/active/set' to update the KB with the result of
 498 the reconfiguration plan and which components are active.

⁷ The */diagnostics* topic is a standard topic for publishing system diagnosis information within the ROS ecosystem. See ROS REP 107 for more information.

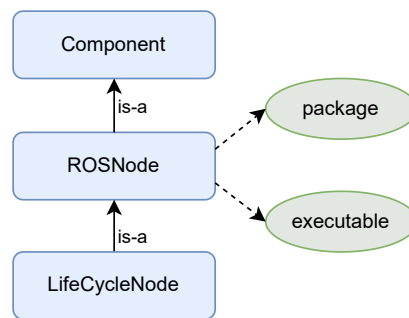


Figure 4. ROS specific knowledge

6.2.5 Task decision layer

ROSA's implementation provides an integration for the task decision layer for both PDDL-based planners and BTs, which are implemented leveraging the PlanSys2 (Martín et al., 2021) and the BehaviorTree.CPP⁸ packages, respectively.

Planning: To enable task decision-making and execution with PDDL-based planners in combination with ROSA, the planner and plan executor must consider the runtime feasibility of performing the robot's actions as inferred by the KB component. This work maps the action status from ROSA's knowledge model to PDDL by capturing whether the action's status is feasible as a PDDL predicate of the form "*action_feasible ?action*" and using it as a precondition to select the respective action. An example can be seen in Listing 4 where the action *my_action* can only be selected when it does not have an 'unfeasible' status in the KB. At runtime, if an action becomes unfeasible during execution, the plan executor triggers re-planning to generate a new action plan. This results in task execution adaptation and, if the newly selected actions require a different architectural configuration, also in architectural adaptation, i.e., TACA.

```

5131 (:durative-action my_action
5142   :parameters (?a – action ...)
5153   :duration (...)
5164   :condition (and
5175     (over all (my_action_action ?a))
5186     (over all (action_feasible ?a))
5197     ...
5208   )
5219   :effect (and ...)
5220 )
523
  
```

Listing 4. PDDL formulation example for ROSA

To handle the interaction between PlanSys2 and ROSA's KB, this work provides a custom ROS 2 node called *RosaPlanner* and a custom PlanSys2 action called *RosaAction*. The *RosaPlanner* is responsible for querying the KB and updating the PDDL problem formulation with information on whether the ROSA actions are feasible or not using the aforementioned "*action_feasible ?action*" PDDL predicate. The

⁸ <https://www.behaviortree.dev/>

528 *RosaAction* action is responsible for querying the KB to request or cancel an *Action* when the execution
 529 of an action starts or finishes. Each PlanSys2 action that should be managed by ROSA should derive from
 530 *RosaAction*, and it should implement the logic for the specific action execution.

531 *Behavior trees*: To enable task decision-making and execution with BTs in combination with ROSA,
 532 the BTs must consider the runtime feasibility of performing the robot's actions as inferred by the KB
 533 component. This work proposes adding before action nodes a condition node that queries the KB to ask
 534 whether the following action is feasible. The proposed pattern is depicted in Figure 5, where the action
 535 *MyAction* would only be executed when its status in the KB is not 'unfeasible'.

536 To enable the use of the BehaviorTree.CPP package to implement BTs for ROSA and abstract away the
 537 interactions with the KB, this work implements a reusable custom condition node called *IsActionFeasible*
 538 and a custom action node called *RosaAction*. The condition node queries the KB to check whether an
 539 *Action* is feasible before selecting it to be executed, and the action node queries the KB to request or
 540 cancel an *Action* when the execution of an action starts or finishes, respectively.

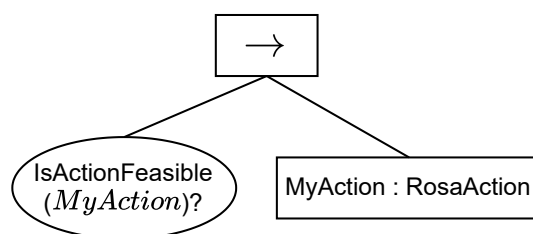


Figure 5. BT pattern for TACA with ROSA. The *IsActionFeasible* condition node takes the action name as a parameter. The *MyAction* node derives from the proposed *RosaAction* node and implements the action execution. The action names in the BT must match the names defined in the KB.

7 EVALUATION

541 This section evaluates ROSA to answer the following questions:

- 542 • *Feasibility*: Is it feasible to use ROSA to enable runtime TACA in ROS 2-based robotic systems?
- 543 • *Performance*: How does ROSA perform compared to other managing subsystems for ROS 2-based
 544 systems?
- 545 • *Reusability*: To what extent can ROSA's knowledge model capture the knowledge required for TACA?
- 546 • *Development effort*: What is the development effort of using ROSA for adding adaptation to different
 547 robotic systems and how does it compare to other approaches?
- 548 • *Development effort scalability*: How does the development effort of using ROSA for adding adaptation
 549 to robotic systems scale for more complex systems?

550 7.1 Experimental design

551 7.1.1 Feasibility

552 To evaluate the feasibility of applying ROSA at runtime to enable TACA in ROS 2-based robotic systems,
 553 it was applied to the SUAVE exemplar described in Section 2. SUAVE was selected since, to the best of
 554 our knowledge, it is the only ROS 2-based open-source exemplar for self-adaptive robotic systems.

7.1.2 Performance

To evaluate ROSA's performance, metrics were collected with the SUAVE exemplar. The metrics collected were the ones available in SUAVE, *search time* and *distance of the pipeline inspected*, in addition to the *reaction time* metric introduced in this paper. For the original use case, the experiments were performed with no managing subsystem, with a BT managing system, with Metacontrol⁹, and with ROSA. For the extended use case, the experiments were performed with a BT managing system and with ROSA.

7.1.3 Reusability

To evaluate to what extent ROSA's knowledge model can capture the knowledge required for TACA, it was used to model the TACA scenarios presented by Cámara et al. (2020) and Braberman et al. (2017)¹⁰, and we showcase how the captured knowledge can be exploited to enable TACA. The experimental setup for both scenarios is not publicly available. Thus, it was not possible to apply ROSA at runtime to the simulated environments they used.

7.1.4 Development effort

To evaluate the development effort of using ROSA to enable adaptation in robotic systems, we analyze the number of elements contained in the ROSA model created to solve the use cases described in this paper. Furthermore, we compare it to the number of elements modeled with the BT approach to solve the SUAVE use case.

7.1.5 Development effort scalability

To analyze how ROSA's development effort scales for more complex use cases, we showcase how many elements must be modeled in ROSA's model to include structural and parameter adaptation in a hypothetical adaptation scenario.

The experimental setup for the *Feasibility* and *Performance* evaluation is open source and reproducible, it can be found at https://github.com/kas-lab/suave_rosa. The models designed to evaluate *Reusability* can be found at https://github.com/kas-lab/rosa_examples.

7.2 Feasibility

7.2.1 Experimental setup

To solve the adaptation scenarios of the SUAVE exemplar with ROSA, the model depicted in Figure 6 was created.

Thruster failure (U_1) was solved with structural adaptation by including two possible function designs of the *maintain motion* function. *Runtime behavior*: when a thruster fails, the *maintain* function design status is set to unfeasible (see Figure 3d), and it cannot be selected anymore. Then, the *recover* function design is selected, and the *recover thruster node* component is activated. If all thrusters are recovered, the *maintain* function design status becomes feasible and is selected again.

Changing water visibility (U_2) was addressed with parameter adaptation by including three component configurations for the *generate spiral node*, each representing a different altitude for searching for the pipeline. Furthermore, a *water visibility* constraint was added to each configuration, representing

⁹ Metacontrol is the only managing subsystem packaged with SUAVE. The Metacontrol implementation for this use case is described in detail in the SUAVE paper (Silva et al., 2023).

¹⁰ To the best of our knowledge, these are the only two scenarios in the literature in which the authors explicitly claim the need for TACA.

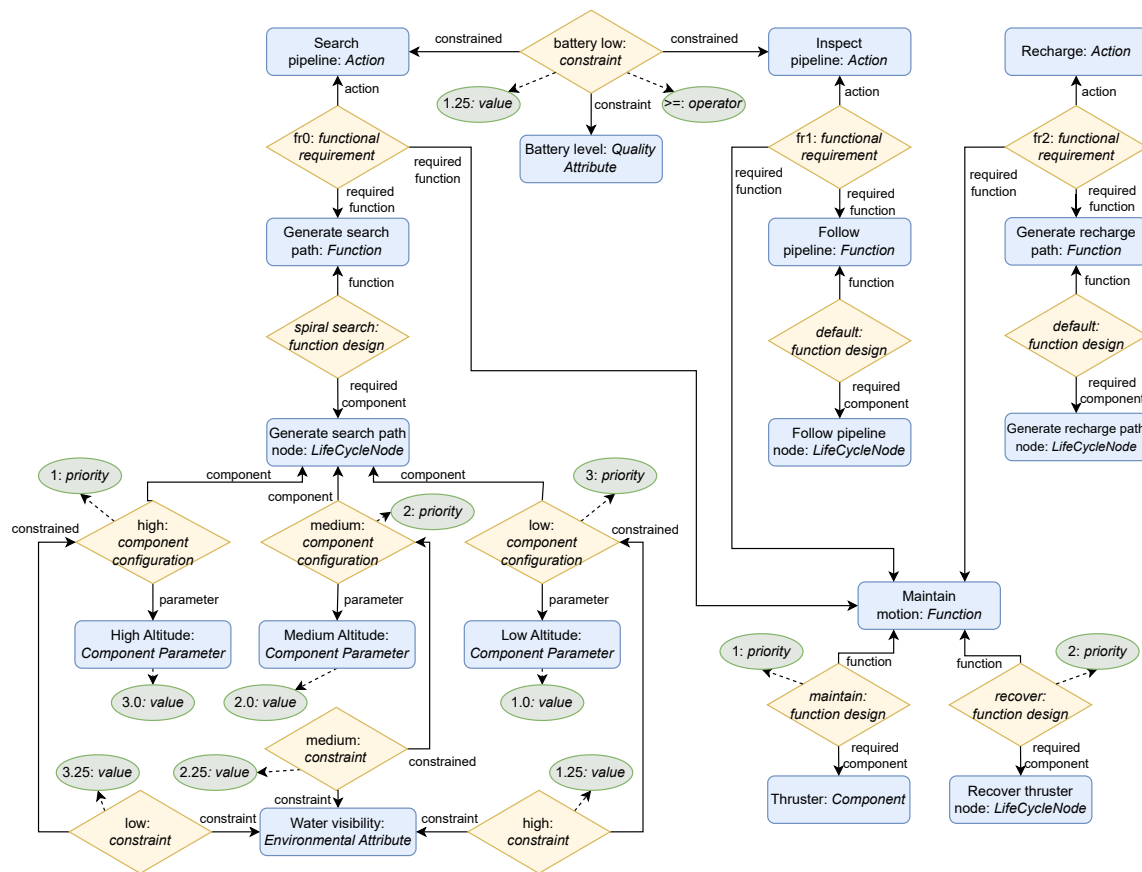


Figure 6. ROSA model for SUAVE.

the minimum water visibility in which the configuration can be used. *Runtime behavior*: if the measured water visibility is higher than 3.25, the component configuration *High* is selected since it has priority number one. If the water visibility drops below 3.25, its constraint status is set to violated (see Figure 3a), and, consequently, its status is set to unfeasible (see Figure 3b). Depending on the water visibility, the component configuration *Medium* or *Low* is then selected. If the water visibility increases again above 3.25, the *High* component configuration status becomes feasible and is selected.

Critical battery level (U_3), occurring only in the extended SUAVE use case, was solved with TACA by extending the knowledge model with a *recharge* action, and a *battery level* constraint to the *search pipeline* and *inspect pipeline* actions, representing the minimum battery level at which they can be selected. *Runtime behavior*: when the battery level drops below 0.25, the status of both the *search pipeline* and *inspect pipeline* actions is set to unfeasible, and the task decision layer cannot select them anymore. Therefore, the task decision layer selects the *recharge* action which also triggers structural adaptation.

Note that extending the solution to solve SUAVE's extended version only required the inclusion of additional knowledge of the *recharge* action and the *constraint* to the *search pipeline* and *inspect pipeline* actions. This demonstrates that an existing application modeled with ROSA can easily be extended to solve additional adaptation scenarios.

To apply ROSA in simulation to the SUAVE exemplar, the knowledge model depicted in Figure 6 was implemented with TypeDB (see an example in Listing 5), and the AUV's mission was implemented with the BT depicted in Figure 7 as well with the PDDL formulation partially shown in Listing 6.

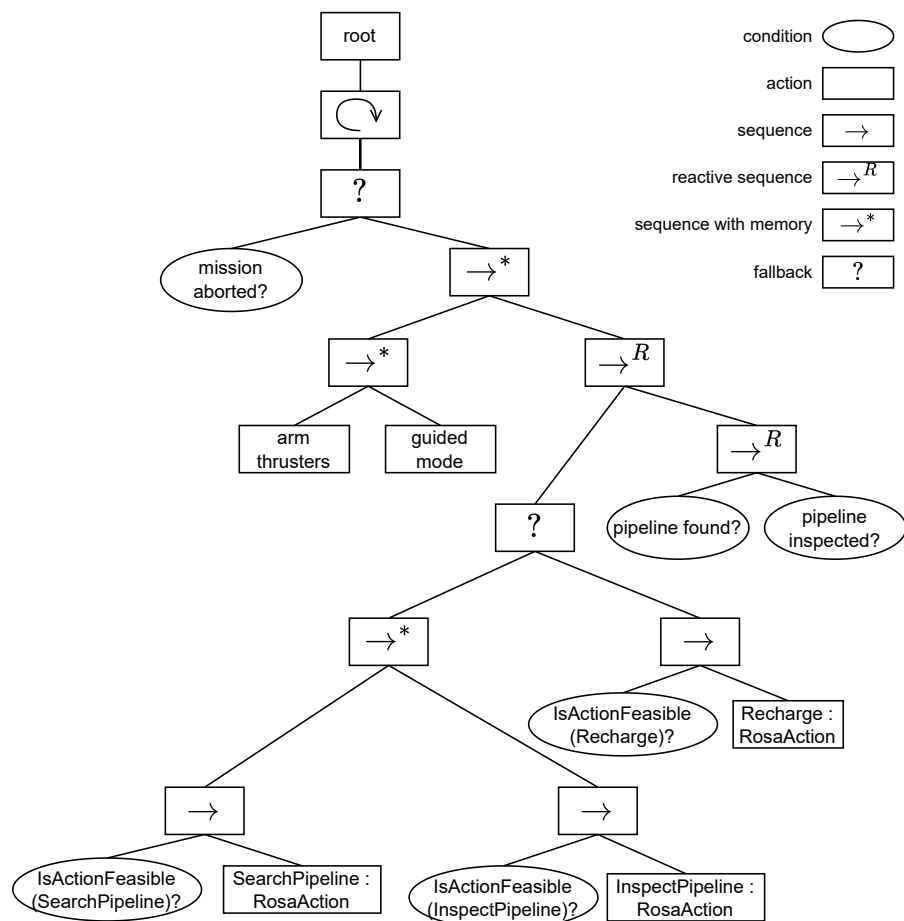


Figure 7. Behavior tree for the extended SUAVE use case

```

6111 # Action search pipeline
6122 $a_search_pipeline isa Action, has action-name
6133   'search_pipeline';
6144
6155 $f_generate_search_path isa Function, has function-name
6166   'generate_search_path';
6177
6188 # functional-requirement relationship
6199 (action: $a_search_pipeline,
6200  required-function: $f_generate_search_path,
6201  required-function: $f_maintain_motion)
6222 isa functional-requirement;

```

Listing 5. Snippet of SUAVE's use case implementation in TypeDB

624 7.2.2 Result

625 During the mission execution, the AUV was able to overcome all uncertainties: adapting to thruster
 626 failures (U_1) with structural adaptation, to changing water visibility (U_2) with parameter adaptation, and to
 627 an unexpected drop in the battery level (U_3) with TACA, demonstrating the feasibility of using ROSA to
 628 enable runtime TACA in ROS 2-based robotic systems.

```

1 (:durative-action search_pipeline
2   :parameters (?a – action ?p – pipeline ?r – robot)
3   :condition (and
4     (over all (robot_started ?r))
5     (over all (search_pipeline_action ?a))
6     (over all (action_feasible ?a))
7   )
8   :effect (and
9     (at end(pipeline_found ?p))
10  )
11 )
12 ...

```

Listing 6. Snippet of the PDDL domain formulation for SUAVE containing the search pipeline action definition.

7.3 Performance

7.3.1 Experimental setup

To evaluate ROSA's performance, the SUAVE exemplar was configured with the same parameters as described in the SUAVE paper (Silva et al., 2023). In addition, for the extended use case, the battery was set to discharge within 200 seconds, and the *search pipeline* and *inspect pipeline* actions were set to require at least 25% of battery to be performed.

7.3.2 Results

The results obtained are shown in Table 6. The different managed subsystems had similar performance despite the difference in their reaction time. Furthermore, since the performance with ROSA was better than without a managing subsystem and close to the other managing subsystems, it can be considered as additional evidence of the feasibility of applying it at runtime to enable self-adaptation in robotic systems.

Table 6. Mission results

Table 6: Mission results								
Managing system	Number of runs	Search time (s)		Distance inspected (s)		Mean reaction time (s)		
		Mean	Std dev	Mean	Std dev	U1	U2	U3
SUAVE								
None	100	174.75	36.00	33.20	13.49	N/A	N/A	N/A
BT	100	84.09	26.41	62.70	7.78	0.08	0.10	N/A
Metacontrol	100	89.24	35.57	60.57	11.17	1.55	0.82	N/A
ROSA	100	85.11	32.48	60.76	10.29	1.24	1.57	N/A
SUAVE extended								
BT	100	94.37	34.92	20.88	3.81	0.07	0.10	1.09
ROSA	100	92.75	35.92	18.97	3.38	1.39	1.67	2.50

7.4 Reusability

7.4.1 Autonomous ground vehicle use case (Cámara et al., 2020)

In this scenario, an AGV has to navigate from an initial to a goal position in a graph-like environment while facing uncertainties such as component failures, corridors with obstacles, and changing light conditions.

The AGV has distinct architectural variants available to solve navigation. It has three localization algorithms (AMCL, MRPT, or aruco) and three sensing components (camera, lidar, or Kinect). However, there are some restrictions on how they can be combined. The AMCL and MRPT algorithms can only be

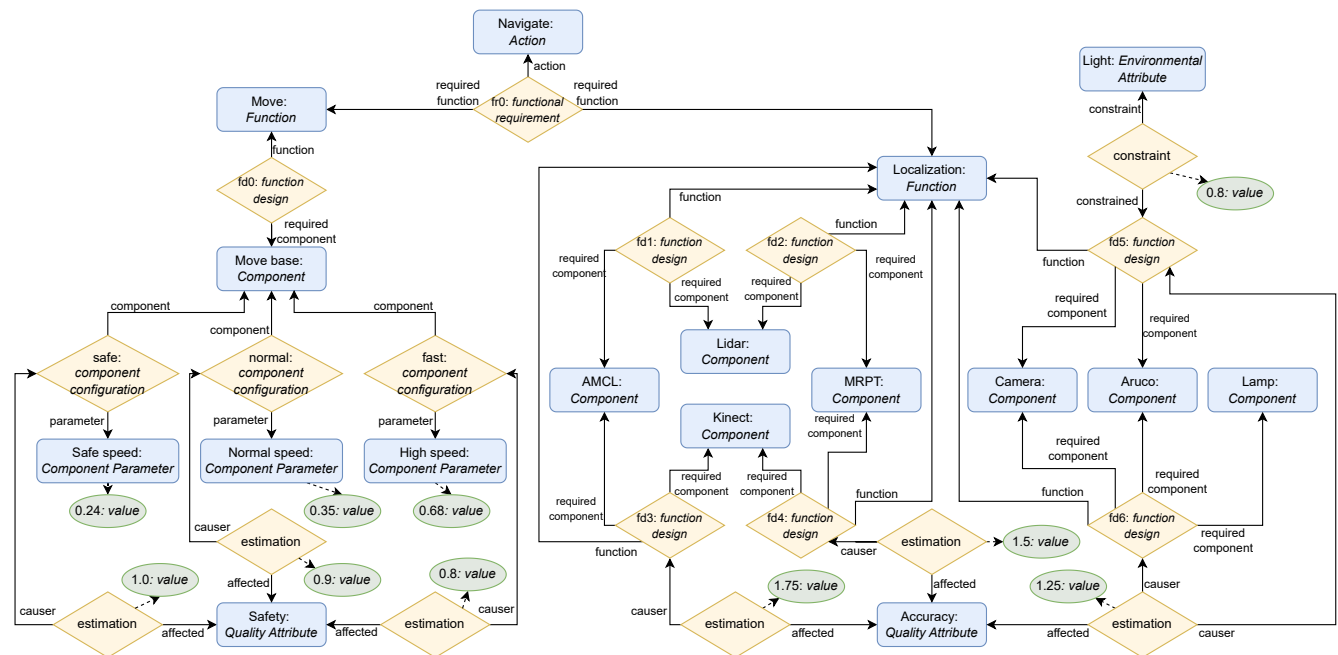


Figure 8. ROSA model for the AGV use case (Cámara et al., 2020)

combined with lidar or Kinect, and the aruco algorithm can only be combined with a camera. In low-light conditions, the camera can only be used with a lamp. Furthermore, the robot can move at three different speeds. Each configuration has a different energy cost, safety, and accuracy level. This scenario can be solved with ROSA with the knowledge model depicted in Figure 8¹¹.

When navigating, the robot performs adaptation by selecting which corridors it needs to go through given its feasible configurations, and by selecting a suitable architecture configuration for each corridor it goes through. For example, to go from point *A* to *B*, the AGV can go directly through a corridor with obstacles *C1* or through corridors *C4* → *C3* → *C2* without obstacles. Ideally, the AGV should go through *C1* as it is the shortest path. Considering that the Kinect and AMCL combination is the only one with enough accuracy to go through a corridor with obstacles, in the case that the Kinect fails, the robot needs to perform TACA by adapting its task plan to go through *C4* → *C3* → *C2* while simultaneously adapting its architecture, e.g., to use the lidar as its sensing component.

7.4.2 Unmanned aerial vehicle use case (Braberman et al., 2017)

In this scenario, a UAV has to search for samples in a predefined area and analyze them. To accomplish this mission, the UAV can perform the actions (*A1*) *search for samples*, (*A2*) *pick up and analyze samples*, (*A3*) *analyze samples on site*, (*A4*) *return to base and recharge*, and (*A5*) *land and fold gripper*. The analyze action *A2* performs a better analysis than *A3*, however, it consumes more battery. Furthermore, action *A2* requires a *gripper*, while *A3* requires an *infra-red camera*. When operating, the UAV might run out of battery, and its gripper might fail.

There are three adaptation scenarios in this use case: (1) if the battery level is insufficient to perform *A1*, the UAV must perform *A4*; (2) if the battery level is insufficient to perform *A2* but it is still sufficient to perform *A3*, the UAV must perform *A3*; (3) if the *gripper* fails while performing *A2*, the UAV must perform *A3*. Before transitioning from *A2* to *A3*, the UAV must first perform *A5*.

¹¹ The accuracy estimation for *fd1* and *fd2* and all energy estimations are omitted from the figure to improve readability

Adaptation scenarios 1 and 2 can be solved with ROSA’s knowledge model by capturing the battery level as a `Quality Attribute` and using it as a `constraint` for actions *A1* and *A2*. Adaptation scenario 3 can be solved by capturing that the `Function` required by *A2* requires a *gripper* component. Furthermore, the task decision layer is responsible for guaranteeing that *A3* is only performed when *A5* is finished.

7.4.3 Results

This evaluation demonstrates that in addition to SUAVE, the knowledge required for the adaptation logic to solve the AGV and UAV use cases can be captured with ROSA’s knowledge model. This indicates that ROSA’s knowledge model can be used to capture the knowledge required for TACA in adaptation scenarios similar to the ones presented. Furthermore, it shows that all entities and relationships contained in the proposed knowledge model had to be used to model the aforementioned adaptation scenarios, supporting their inclusion in the knowledge model.

7.5 Development effort

7.5.1 Experimental Setup

To measure ROSA’s development effort for each use case, we count the number of entities and relationships contained in the models created for the use cases presented. To measure the BT managing system development effort for SUAVE, we count the number of nodes contained in the BTs created to solve it, and the number of modes and parameters included in the System Modes’ configuration file already packaged in SUAVE. In the remainder of this paper, we denote the elements modeled in both approaches as *overhead*.

7.5.2 Results

The overhead for both approaches can be seen in Table 7. Although it is not possible to make a straightforward comparison between the development efforts of both approaches using the observed overheads since the difficulty of modeling an element using the different modeling techniques is subjective, analyzing the reason for the observed overheads provides insights for comparing the development efforts of both approaches.

Table 7. Development effort of using ROSA and BTs as managing subsystems

ROSA			
Use case	Entities	Relations	Total
SUAVE	18	12	30
SUAVE extended	22	16	38
AGV	18	36	54
UAV	24	16	40
Behavior tree			
Use case	BT	System modes	Total
SUAVE	27	30	57
SUAVE extended	34	38	72

ROSA: In TypeDB, each entity and relationship is inserted in the KB with TypeQL queries such as the ones presented in Listing 5. Thus, the total number of queries that the roboticist must define to solve an adaptation scenario is equal to the sum of the number of entities and relationships contained in the model, with a clear separation between the task and adaptation logic.

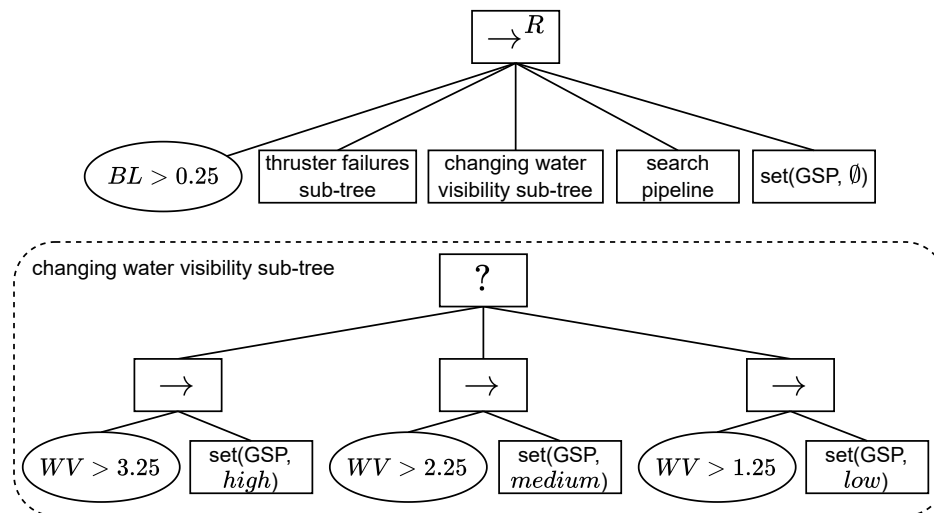


Figure 9. Snippet of the BT used to capture SUAVE's task and adaption logic. *WV* and *BL* represent the measured water visibility and battery level, respectively. *GSP*, *MM*, and *FP* represent SUAVE's *generate search path node*, *maintain motion node*, and *follow pipeline node*, respectively. The *set* action changes the system's modes.

699 *BT managing system:* The BT used to model SUAVE's task logic without adaptation contains 10 nodes,
 700 and the BT used for the extended use case contains 16 nodes. These values were deducted from the
 701 development effort metric for the BT managing system since they are independent of the adaptation
 702 problem.

703 Figure 9 depicts the pattern used to model SUAVE's *search pipeline* action and its related adaptations¹².
 704 As can be seen, there is no separation between the task and adaptation logic, which is the main limitation
 705 of using BTs in comparison to using ROSA to model the adaptation logic. The coupling of both logics
 706 hinders the reusability of the approach as another system with the same task logic but different adaptation
 707 logic, or vice-versa, cannot reuse the existing BTs. In addition, when any changes are made to the task or
 708 adaptation problems, it will most likely require changes to parts of the BT that are not necessarily related to
 709 the changes introduced. Furthermore, it makes the modeling process more difficult as the roboticist needs
 710 to consider both problems simultaneously when modeling the BTs.

711 7.6 Development effort scalability

712 7.6.1 Experimental setup

713 To evaluate how ROSA's development effort scales for more complex use cases, we analyze how the
 714 development effort of a base scenario grows with the addition of new actions and adaptations. The growth
 715 for adding actions and adaptations depends on the specific application. Thus, we make the following
 716 assumptions to generalize and simplify the analysis of adding adaptation.

717 **ASSUMPTION 1.** *Every action requires one function that has only one configuration available consisting*
 718 *of a single component with no parameters.*

719 **ASSUMPTION 2.** *Every Component is a ROSNode containing one package and one executable*
 720 *attribute; every ROSNode has one component configuration with one Component Parameter;*

¹² The full BT can be found in the SUAVE repository.

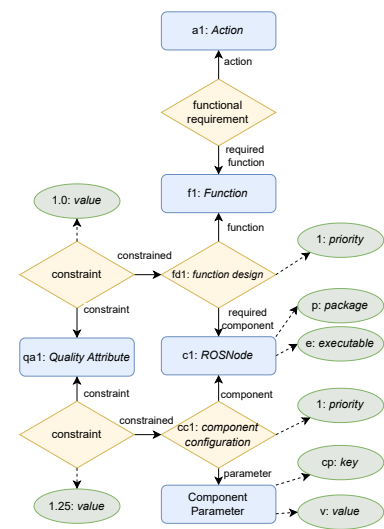


Figure 10a. Base model

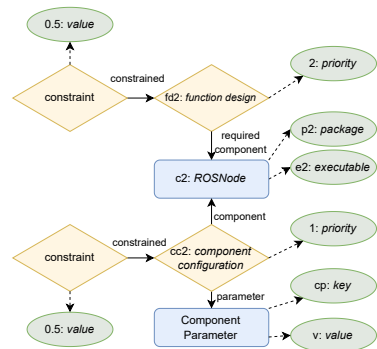


Figure 10b. Structural adaptation

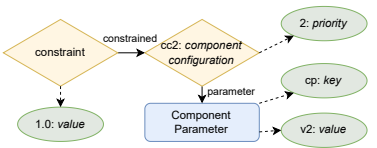


Figure 10c. Parameter adaptation

Figure 10. Minimum knowledge required to model adaptation given Assumptions 1 and 2. The name attribute is depicted as *name: Entity* or *name: Relationship* inside each diamond and rectangle shape, respectively.

7.6.2 Results

Consider a base scenario where the robot has only one action and complies with Assumptions 1 and 2. The ROSA model to solve it contains 10 elements and is depicted in Figure 10a.

To add structural adaptation to the base scenario (given Assumption 2), it is necessary to add the elements depicted in Figure 10b to the model. This results in a minimum overhead of 6 elements for each structural adaptation. To add parameter adaptation to the base scenario (given Assumption 2), it is necessary to add the elements depicted in Figure 10c to the model. This results in a minimum overhead of 3 elements for each parameter adaptation.

In conclusion, given Assumptions 1 and 2, the total overhead per action can be defined as $10 + 6 * n_{sa} + 3 * n_{pa}$, where n_{sa} and n_{pa} represent the number of structural and parameter adaptations for the action, respectively. This indicates that the ROSA model grows linearly with the number of actions and adaptations, which is made possible by the clear separation of the task and adaptation logic.

8 CONCLUSIONS AND FUTURE WORK

This work proposed ROSA, a knowledge-based solution for task-and-architecture co-adaptation in robotic systems that promotes reusability, extensibility, and composability. Reusability was achieved by proposing a knowledge model that can capture the knowledge required for TACA and using it at runtime to reason about adaptation. Extensibility and composability were achieved with an architectural design that allows ROSA's components to be stateless and self-contained. The feasibility of using ROSA in robotic systems at runtime was demonstrated by applying it in simulation to the SUAVE exemplar. ROSA's reusability was demonstrated by using it to model different self-adaptive robotic systems and showing that it can capture all relevant knowledge for adaptation necessary for these use cases. Furthermore, ROSA's development effort and its scalability were demonstrated for the use cases presented in this paper and for a hypothetical scenario.

ROSA modular architecture has been designed to provide reuse and extensibility of the framework by future works applying self-adaptation principles in robotics architectures. For example, the current ROSA implementation supports integration with robotics architectures using planning or behavior tree solutions for the task deliberation layer, but it would be interesting to extend it to support other decision-making methods, such as state machines or Markov decision processes.

As a future work, we intend to integrate learning in ROSA. For example, machine learning methods could be explored to update at runtime the `Quality Attribute` and `Environmental Attribute` estimations and constraints values. Another possibility is learning that constraints and estimations exist without prior knowledge, i.e., learning that the relationship itself should be modeled. ROSA's interfaces to manipulate the KB at runtime could be exploited for this end.

CONFLICT OF INTEREST STATEMENT

The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

AUTHOR CONTRIBUTIONS

GRS: Conceptualization, Data Curation, Investigation, Methodology, Software, Writing – original draft, Writing – review & editing. JP: Conceptualization, Writing – review & editing. SLTT: Conceptualization, Writing – review & editing. EBJ: Conceptualization, Writing – review & editing. CHC: Conceptualization, Writing – review & editing.

FUNDING

This work was supported by the European Union’s Horizon 2020 Framework Programme through the MSCA network REMARO (Grant Agreement No 956200).

REFERENCES

- (2024). IEEE Approved Draft Standard for Robot Task Representation. *IEEE P1872.1/D5*, October 2023 , 1–35
- Alami, R., Chatila, R., Fleury, S., Ghallab, M., and Ingrand, F. (1998). An architecture for autonomy. *The International Journal of Robotics Research* 17, 315–337. doi:10.1177/027836499801700402
- Alberts, E., Gerostathopoulos, I., Malavolta, I., Hernández Corbato, C., and Lago, P. (2025). Software architecture-based self-adaptation in robotics. *Journal of Systems and Software* 219, 112258. doi:https://doi.org/10.1016/j.jss.2024.112258
- Antoniou, G. and van Harmelen, F. (2004). Web Ontology Language: OWL. In *Handbook on Ontologies*, eds. S. Staab and R. Studer (Springer), International Handbooks on Information Systems. 67–92. doi:10.1007/978-3-540-24750-0_4
- Barnett, W., Cavalcanti, A., and Miyazawa, A. (2022). Architectural modelling for robotics: Roboarch and the cortex example. *Frontiers in Robotics and AI* 9, 991637
- Beetz, M., Mösenlechner, L., and Tenorth, M. (2010). CRAM — A Cognitive Robot Abstract Machine for everyday manipulation in human environments. In *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 1012–1017. doi:10.1109/IROS.2010.5650146
- Board, S. E. (2023). *The Guide to the Systems Engineering Body of Knowledge (SEBoK)*, v. 2.8, R.J. Cloutier (Editor in Chief). Hoboken, NJ: The Trustees of the Stevens Institute of Technology. Accessed 9-October-2023. BKCASE is managed and maintained by the Stevens Institute of Technology Systems Engineering Research Center, the International Council on Systems Engineering, and the Institute of Electrical and Electronics Engineers Systems Council
- Bozhinoski, D., Oviedo, M. G., Garcia, N. H., Deshpande, H., van der Hoorn, G., Tjerngren, J., et al. (2022). MROS: runtime adaptation for robot control architectures. *Advanced Robotics* 36, 502–518. doi:10.1080/01691864.2022.2039761
- Bozhinoski, D. and Wijkhuizen, J. (2021). Context-based navigation for ground mobile robot in semi-structured indoor environment. In *2021 Fifth IEEE International Conference on Robotic Computing (IRC)*. 82–86. doi:10.1109/IRC52146.2021.00019
- Braberman, V., D’Ippolito, N., Kramer, J., Sykes, D., and Uchitel, S. (2017). An extended description of MORPH: A reference architecture for configuration and behaviour self-adaptation. In *Software Engineering for Self-Adaptive Systems III. Assurances*, eds. R. de Lemos, D. Garlan, C. Ghezzi, and H. Giese (Cham: Springer International Publishing), Lecture Notes in Computer Science, 377–408. doi:10.1007/978-3-319-74183-3_13

- Cámara, J., Schmerl, B., and Garlan, D. (2020). Software Architecture and Task Plan Co-Adaptation for Mobile Service Robots. In *Proc. 15th Intl. Symposium on Software Engineering for Adaptive and Self-Managing Systems* (New York, NY, USA: ACM), SEAMS'20, 125–136. doi:10.1145/3387939.3391591
- Carreno, Y., Scharff Willners, J., Petillot, Y. R., and Petrick, R. (2021). Situation-Aware Task Planning for Robust AUV Exploration in Extreme Environments. In *Proc. IJCAI Workshop on Robust and Reliable Autonomy in the Wild*
- Chen, P. P. (1976). The entity-relationship model - toward a unified view of data. *ACM Trans. Database Syst.* 1, 9–36. doi:10.1145/320434.320440
- Colledanchise, M. and Ögren, P. (2018). *Behavior trees in robotics and AI: An introduction* (CRC Press)
- Dorn, C. and Pribadi, H. (2023). Type theory as a unifying paradigm for modern databases. In *Proc. 32nd Intl. Conf. on Information and Knowledge Management (CIKM 2023)*, eds. I. Frommholz, F. Hopfgartner, M. Lee, M. Oakes, M. Lalmas, M. Zhang, and R. L. T. Santos (ACM), 5238–5239. doi:10.1145/3583780.3615999
- Dorn, C. and Pribadi, H. (2024). TypeQL: A type-theoretic & polymorphic query language. In *2024 ACM SIGMOD/PODS International Conference on Management of Data. To be published.* (ACM)
- Garcia, S., Menghi, C., Pelliccione, P., Berger, T., and Wohlrab, R. (2018). An architecture for decentralized, collaborative, and autonomous robots. In *2018 IEEE International Conference on Software Architecture (ICSA)*. 75–7509. doi:10.1109/ICSA.2018.00017
- [Dataset] Ghallab, M., Howe, A., Knoblock, C., Mcdermott, D., Ram, A., Veloso, M., et al. (1998). PDDL—The Planning Domain Definition Language
- Gherardi, L. and Hochgeschwender, N. (2015). RRA: Models and tools for robotics run-time adaptation. In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1777–1784. doi:10.1109/IROS.2015.7353608
- Hamilton, J., Stefanakos, I., Calinescu, R., and Cámara, J. (2022). Towards adaptive planning of assistive-care robot tasks. In *Proc. 4th Intl. Workshop on Formal Methods for Autonomous Systems (FMAS 2022)*, eds. M. Luckcuck and M. Farrell. vol. 371 of *EPTCS*, 175–183. doi:10.4204/EPTCS.371.12
- Hernández, C., Bermejo-Alonso, J., and Sanz, R. (2018). A Self-Adaptation Framework Based on Functional Knowledge for Augmented Autonomy in Robots. *Integrated Computer-Aided Engineering* 25, 157–172
- Hochgeschwender, N., Schneider, S., Voos, H., Bruyninckx, H., and Kraetzschmar, G. K. (2016). Graph-based software knowledge: Storage and semantic querying of domain models for run-time adaptation. In *Proc. Intl. Conf. on Simulation, Modeling, and Programming for Autonomous Robots (SIMPAN 2016)* (IEEE), 83–90. doi:10.1109/SIMPAN.2016.7862379
- Kazhoyan, G., Stelter, S., Kenfack, F. K., Koralewski, S., and Beetz, M. (2021). The robot household marathon experiment. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*. 9382–9388. doi:10.1109/ICRA48506.2021.9560774
- Kephart, J. O. and Chess, D. M. (2003). The vision of autonomic computing. *Computer* 36, 41–50
- Kortenkamp, D., Simmons, R., and Brugali, D. (2016). *Robotic Systems Architectures and Programming* (Cham: Springer International Publishing). 283–306. doi:10.1007/978-3-319-32552-1_12
- Kotseruba, Iuliia, Tsotsos, and K., J. (2018). 40 years of cognitive architectures: core cognitive abilities and practical applications. *Artificial Intelligence Review* 53, 17–94. doi:10.1007/s10462-018-9646-y
- Lotz, A., Inglés-Romero, J. F., Vicente-Chicote, C., and Schlegel, C. (2013). Managing run-time variability in robotics software by modeling functional and non-functional behavior. In *Enterprise, Business-Process*

- and *Information Systems Modeling*, eds. S. Nurcan, H. A. Proper, P. Soffer, J. Krogstie, R. Schmidt, T. Halpin, and I. Bider (Berlin, Heidelberg: Springer Berlin Heidelberg), 441–455
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. (2022). Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics* 7, eabm6074. doi:10.1126/scirobotics.abm6074
- Martín, F., Clavero, J. G., Matellán, V., and Rodríguez, F. J. (2021). Plansys2: A planning system framework for ros2. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 9742–9749. doi:10.1109/IROS51168.2021.9636544
- Niemczyk, S. and Geihs, K. (2015). Adaptive run-time models for groups of autonomous robots. In *2015 IEEE/ACM 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 127–133. doi:10.1109/SEAMS.2015.21
- Niemczyk, S., Opfer, S., Fredivianus, N., and Geihs, K. (2017). Ice: self-configuration of information processing in heterogeneous agent teams. In *Proceedings of the Symposium on Applied Computing* (New York, NY, USA: Association for Computing Machinery), SAC '17, 417–423. doi:10.1145/3019612.3019653
- Nordmann, A., Lange, R., and Rico, F. M. (2021). System modes - digestible system (re-)configuration for robotics. In *2021 IEEE/ACM 3rd International Workshop on Robotics Software Engineering (RoSE)*. 19–24. doi:10.1109/RoSE52553.2021.00010
- Olivé, A. (2007). *Conceptual modeling of information systems* (Springer). doi:10.1007/978-3-540-39390-0
- Park, Y.-S., Koo, H.-M., and Ko, I.-Y. (2012). A task-based and resource-aware approach to dynamically generate optimal software architecture for intelligent service robots. *Software: Practice and Experience* 42, 519–541
- Sanchez-Lopez, J. L., Suárez Fernández, R. A., Bavle, H., Sampedro, C., Molina, M., Pestana, J., et al. (2016). Aerostack: An architecture and open-source software framework for aerial robotics. In *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*. 332–341. doi:10.1109/ICUAS.2016.7502591
- Silva, G. R., Päßler, J., Zwanepol, J., Alberts, E., Tapia Tarifa, S. L., Gerostathopoulos, I., et al. (2023). SUAVE: An exemplar for self-adaptive underwater vehicles. In *Proc. 18th IEEE/ACM Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS 2023)* (IEEE), 181–187
- Thalheim, B. (1993). Foundations of entity - relationship modeling. *Annals of Mathematics and Artificial Intelligence* 7, 197–256. doi:10.1007/BF01556354
- Thalheim, B. (2000). *Entity-relationship modeling - foundations of database technology* (Springer)
- Valner, R., Vunder, V., Aabloo, A., Pryor, M., and Kruusamäe, K. (2022). TeMoto: A Software Framework for Adaptive and Dependable Robotic Autonomy With Dynamic Resource Management. *IEEE Access* 10, 51889–51907
- Weyns, D. (2020). *An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective* (John Wiley & Sons)
- Weyns, D., Schmerl, B., Grassi, V., Malek, S., Mirandola, R., Prehofer, C., et al. (2013). On Patterns for Decentralized Control in Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010 Revised Selected and Invited Papers*, eds. R. de Lemos, H. Giese, H. A. Müller, and M. Shaw (Berlin, Heidelberg: Springer). 76–107. doi:10.1007/978-3-642-35813-5_4